

APP для МК61s: совместимость и прежняя сборка ABI 2

Версия документа: 06.09.2026

Этот документ описывает пользовательские исполняемые файлы .APP для STM32F401CC и сохранённый путь manifest-сборки ABI 2.

Для новых самостоятельных программ с `main()` используйте отдельное [руководство по C/C++ и ABI 3](#).

Штатный ABI 3 позволяет собирать C/C++ APP без resident ELF/BIN, с фиксированным SRAM-адресом и автоматическим выбором ZX0 либо BCJ + ZX0. Технический контракт: [sdk/portable/README.md](#). Он включён в комплектных сборщиках по умолчанию. Все System APP используют этот формат; ниже также описан совместимый прежний путь ABI 2 (-PortableApps 0).

Модель APP

.APP — единый тип исполняемого контейнера. Поле `kind` внутри заголовка различает:

- пользовательское приложение APPLICATION;
- системные компоненты FOCAL, TinyBASIC, WBMP viewer, Markdown viewer и консоль CHIP-8.

Отдельных расширений .MOD или .MKA нет. Системные файлы имеют канонические пути /System/FOCAL.APP, /System/BASIC.APP, /System/WBMP.APP, /System/MARKDOWN.APP и /System/CHIP8.APP. Пользовательский APP можно хранить под любым допустимым именем в любом каталоге C5.

APP содержит образ для общего 20-КиБ SRAM-overlay. В ABI 3 его адрес фиксирован (0x20000000); связи с прошивкой проходят через таблицы C API. Payload сжат ZX0, при выигрыше размера применяется обратимый Thumb BCJ. В ABI 2 образ связан с точным resident ELF и может быть несжатым. 64-байтовый заголовок содержит ABI, размеры, точку входа, CRC потока и образа, поля привязки resident (нулевые в ABI 3) и необязательную двухбайтовую сигнатуру типа C5 для обработчика файлов.

При установке прошивка проверяет весь контейнер до изменения дерева C5. При запуске она проверяет совместимость, распаковывает образ в overlay, восстанавливает данные и вызывает точку входа. System APP сохраняет кэш между командами; программа и переменные языков живут в отдельной арене.

При первом переходе на ABI 3 обновите прошивку и /System. После этого совместимую прошивку можно обновлять без пересборки APP. Правило точного совпадения .bin и .APP из одного комплекта сохраняется только для ABI 2.

На F401 МК61_ENABLE_LOADABLE_MODULES=1 включается профилем платы автоматически. На штатной F411-сборке загрузчик APP выключен, а системные компоненты остаются во внутренней Flash.

Штатные APP через Arduino IDE

Если нужны штатные FOCAL.APP, BASIC.APP, MARKDOWN.APP, альтернативный ему WBMP.APP или CHIP8.APP, писать manifest или пример пользовательского приложения не требуется. Установите плату МК61s F401 + APP:

```
./tools/mk61-arduino-board.cmd
```

На Windows:

```
tools\mk61-arduino-board.cmd
```

После перезапуска Arduino IDE откройте `code/code.ino`, выберите эту плату, платформу, экран и значения APP у нужных системных компонентов. Verify и Upload оба сначала создают согласованный каталог `binary/`; Upload затем записывает в DFU resident .bin. Каталог System из этого же результата копируется в /System на C5 после запуска прошивки.

WBMP и CHIP-8 требуют графического backend. Для физического UC1609 они работают непосредственно; для mini с LCD1602 необходимо дополнительно включить USB-экран. Графический MARKDOWN.APP сам открывает WBMP, поэтому отдельный WBMP.APP собирается только при выключенном Markdown. Сам

CHIP-8 по умолчанию выключен.

Для этого пути не нужен установленный `arduino-cli` или `Bash` под Windows. Плата использует ARM-компилятор из STM32 Core, Python 3.10+ и нативный C++17-компилятор для упаковщика. Штатные APP используют ABI 3 и выбор ZX0 либо BCJ + ZX0. Полная инструкция находится в [МК61s-mini-Arduino-IDE.md](#).

Штатные APP через ARM toolchain

Для командной сборки все пять системных компонентов вынесены в общий каталог:

```
system_apps/  
├── build.cmd  
├── focal/main.cpp  
├── basic/main.cpp  
├── wbmp/main.cpp  
├── markdown/main.cpp  
└── chip8/main.cpp
```

Эти объединённые `main.cpp` сохранены для ABI 2 (`-PortableApps 0`). В этом режиме каждый `main.cpp` включает реализацию только своего модуля и компилируется в один ARM-объект. Arduino-библиотеки и остальные объектные файлы проекта к нему не долиnkуются: общие вызовы разрешаются из точного resident ELF через `--just-symbols`. Затем `arm-none-eabi-objcopy` извлекает `.module_image`, а нативный `host`-паковщик создаёт контейнер ZX0 с CRC этого же resident BIN. По умолчанию оболочка собирает ABI 3 через `tools/build_portable_app.py`, без такой привязки. Отдельные модули ABI 3 можно собирать и без каталога прошивки; команды приведены в [System APP](#).

Windows-порт `tools/mk61-firmware.cmd` вызывает этот сборщик автоматически для всех включённых FOCAL, TinyBASIC, WBMP, Markdown и CHIP-8. Для ARM-части используются `arm-none-eabi-g++`, `objcopy`, `nm` и `size` из установленного STM32 Core; упаковщик дополнительно требует нативный C++17-компилятор.

Инструмент можно запустить отдельно, если уже имеется каталог одной Arduino- или прямой CMake/GCC-сборки с resident `.elf`, `.bin` и `compile_commands.json`:

```
system_apps\build.cmd ^  
-BuildPath C:\work\mk61-resident ^  
-Focal 1 -Basic 1 -Wbmp 0 -Markdown 1 -Chip8 1
```

По умолчанию файлы появляются в `system_apps\System`; параметр `-OutputDirectory` задаёт другое место. `compile_commands.json` используется не как список готовых APP-объектов, а как надёжный источник версии ARM toolchain, include-путей и флагов конкретной resident-сборки. На fullscreen bitmap backend MARKDOWN.APP обслуживает T2 и I1, включая квалификационный профиль WS0010 100x16; CHIP8.APP обслуживает C1.

Каталог приложения и manifest (ABI 2)

Оставшаяся часть документа описывает прежние пользовательские APP ABI 2: собственную точку входа, manifest, привязку к resident и кэш состояния. Для ABI 3 эти правила заменены startup SDK и свежими `.data/BSS` при каждом запуске, как описано в [новом руководстве](#).

Исходники APP не нужно помещать в `code/` и не нужно добавлять в `build_f401_bundle.sh`. Создайте отдельный каталог:

```
MYAPP/  
├── app.mk61  
├── main.cpp  
├── logic.cpp  
└── include/  
    └── logic.hpp
```

`app.mk61`:

```
format 1  
name MYAPP  
source main.cpp  
source logic.cpp  
file include/logic.hpp
```

Директивы имеют следующий смысл:

Директива	Обязательность	Назначение
format 1	одна	Версия синтаксиса manifest.
name NAME	одна	Basename результата Apps/NAME.APP, до 31 ASCII-байта.
magic XX	необязательно	Двухбайтовая сигнатура типа C5 для FILE_OPEN.
source path.cpp	одна или больше	Единица трансляции, входящая в overlay.
file path	необязательно	Заголовок или .inc, нужный исходникам.

Пути задаются относительно manifest. Они должны быть ASCII, без пробелов, обратной косой черты, абсолютного пути, . и ... Пути сравниваются без учёта регистра, чтобы один проект одинаково собирался на Linux, macOS и Windows. Имя APP использует буквы, цифры, _ и -, не начинается с _ или - и уникально без учёта регистра. Имена CON, PRN, AUX, NUL, COM1--COM9 и LPT1--LPT9 зарезервированы FAT/DOS и отклоняются. Компилируемый файл нельзя маскировать директивой file: все .cpp должны быть явно перечислены как source; .c, .cc, .cxx, .s и .ino текущий SDK не принимает.

magic содержит ровно две ASCII-буквы или цифры с учётом регистра. Например, magic C2 регистрирует обработчик второй консольной платформы. Директива не создаёт новый тип или расширение C5: соответствие расширения, типа и сигнатуры должно уже присутствовать в resident. Обычному запускаемому APP директива не нужна.

Проверить manifest без Arduino-сборки:

```
./tools/build_f401_bundle.sh --check-app-manifests \
--app-manifest path/to/MYAPP/app.mk61
```

Готовый пример находится в examples/loadable-apps/HELLO/.

Точка входа и resident API v1

Сборщик автоматически предоставляет приложению только три SDK-заголовка:

- loadable_app_api.hpp;
- loadable_module_abi.hpp;
- rust_types.h.

Внутренние заголовки resident намеренно не копируются. Поддерживаемая граница между приложением и прошивкой — версионная таблица loadable_app::Api, переданная в argument0. Прямой импорт C++-символов resident не является публичным ABI.

Минимальная точка входа:

```

#include "loadable_app_api.hpp"
#include "loadable_module_abi.hpp"

namespace {

const loadable_app::Api* api;

bool set_api(u32 argument0) {
    const loadable_app::Api* candidate =
        loadable_app::from_argument(argument0);
    if(!loadable_app::compatible(candidate)) return false;
    api = candidate;
    return true;
}

} // namespace

extern "C" __attribute__((used, section(".mk61_module_entry")))
u32 mk61_module_entry(u32 raw_command, u32 argument0,
                      u32, u32, u32) {
    const loadable_module::Command command =
        (loadable_module::Command) raw_command;
    switch(command) {
        case loadable_module::Command::INITIALIZE:
            return set_api(argument0) ? 0 : 1;

        case loadable_module::Command::APPLICATION_RUN: {
            if(!set_api(argument0)) return 1;
            static const char text[] = "HELLO APP";
            api->display_clear();
            api->display_write_utf8(0, 0, text, sizeof(text) - 1);
            return 0;
        }

        default:
            return 0;
    }
}

```

Имя `mk61_module_entry`, C linkage, пять аргументов `u32` и секция `.mk61_module_entry` обязательны. Указатель API передаётся и при `INITIALIZE`, и при `APPLICATION_RUN`; приложение должно проверять `magic`, версию и размер через `loadable_app::compatible()`.

API v1 содержит:

Capability	Возможности
CAP_TIME	<code>millis_ms</code> , обслуживание фоновых задач, безопасная задержка.
CAP_TEXT_DISPLAY	Размер экрана, очистка и UTF-8-текст до 63 байт.
CAP_KEYBOARD	Неблокирующий опрос и ожидание нормализованной клавиши.
CAP_LED	Состояние и асинхронное мигание штатного LED.
CAP_SOUND	Проверенный звуковой сигнал и остановка звука.
CAP_FILES	Длина и потоковое чтение файла C5 по стабильному ID.
CAP_GRAPHICS	Fullscreen-кадр 192x64 и отслеживание смены backend.
CAP_KEY_STATE	Опрос одновременно удерживаемых логических клавиш.

Перед вызовом необязательной группы проверяйте соответствующий `capability`. Логические значения `loadable_app::Key` не зависят от физической раскладки `mini`, `Classic` или `40th`. Неизвестный `scan-code` сохраняется как `KEY_RAW_BASE + scan_code`.

Новые `callbacks` добавлены после прежнего 64-байтового префикса. Старый APP проверяет известный ему меньший `struct_size`, а новый обязан убедиться, что хвост нужной группы присутствует. Графический кадр имеет размер 1536 байт и `page-major` разметку 192x64: последовательные 192 байта описывают одну

полосу высотой восемь пикселей.

API v1 намеренно не содержит записи C5, низкоуровневого SPI, USB, управления тактированием и прерываниями. CAP_FILES даёт только чтение переданного файла. Это уменьшает вероятность повредить файловую систему из обычного приложения.

APP как обработчик типа файла

Ненулевой `handled_type_magic` в заголовке APP регистрирует обработчик двухбайтового типа. Это не `content magic` и не первые байты пользовательского файла. Например, сырой ROM `.ch8` имеет тип CHIP8 и сигнатуру C1, но сам ROM начинается сразу с `opcode`.

Проводник вызывает зарегистрированный APPLICATION общей командой:

```
case loadable_module::Command::FILE_OPEN: {
    if(!set_api(argument0)) {
        return (u32) loadable_module::FileOpenResult::RUNTIME_ERROR;
    }
    const u32 file_id = argument1;
    // api->file_size(file_id), api->file_read(...)
    return (u32) loadable_module::FileOpenResult::OK;
}
```

`argument1` содержит стабильный ID файла C5. System APP получает ту же команду, но `argument0` для него равен нулю: штатные модули используют `resident`-импорты. Обработчик возвращает `OK`, `INVALID_FILE`, `UNSUPPORTED_DISPLAY`, `BUSY`, `IO_ERROR` либо `RUNTIME_ERROR`, чтобы Проводник и терминальная команда `open` показывали одинаковый результат.

Приоритет регистрации: встроенный обработчик, канонический System APP, затем пользовательский APPLICATION. Два пользовательских APP с одинаковым `magic` считаются неоднозначной конфигурацией и не должны выбираться по порядку файлов. Техническое задание и таблица семейств приведены в [MK61s-mini-CHIP8.md](#).

Время жизни

После распаковки нового образа `resident` один раз вызывает `INITIALIZE`. При каждом открытии того же кэшированного файла вызывается `APPLICATION_RUN`. Загрузка другого APP перезаписывает `overlay`; при возврате к первому файлу его образ загружается и инициализируется заново.

Для APP-обработчика вместо `APPLICATION_RUN` при каждом открытии целевого файла вызывается `FILE_OPEN`. Его статическое состояние имеет то же время жизни: сохраняется до загрузки другого APP, но не должно использоваться как замена переданному `file ID`.

В один момент в SRAM находится один APP. Вызов синхронный и должен вернуться. Долгий цикл обязан регулярно вызывать `api->service()` либо использовать `api->delay_ms()`. Нельзя вложенно запускать другой APP.

Статические POD-переменные допустимы. BSS обнуляется при загрузке, но не перед каждым повторным `APPLICATION_RUN`. Если состояние должно сбрасываться при каждом запуске, сделайте это в обработчике команды.

Ограничения и доверие

- `.text`, `.rodata`, инициализированные данные и BSS вместе занимают не более 20 КиБ;
- контейнер занимает от 64 до 20 544 байт;
- стек общий с `resident` и не входит в `overlay`: избегайте больших локальных массивов и глубокой рекурсии;
- `linker` запрещает глобальные конструкторы и деструкторы;
- исключения, `RTTI`, `malloc` и `new` не входят в поддерживаемый контракт;
- код собирается для Cortex-M4 Thumb, hard-float ABI, с `-Os -flto`;
- оптимальная ZX0-упаковка выполняется на компьютере; распаковщик в `resident` не меняется от уровня сжатия.

APP — нативный код без песочницы и должен считаться доверенным. Минимальный SDK не даёт ему случайно подключить внутренний C5 API, но намеренный доступ к регистрам или произвольным адресам нельзя сделать безопасным проверкой контейнера. Код, который напрямую перенастраивает SPI flash, USB, MPU, тактирование или прерывания, находится вне поддерживаемой модели и способен нарушить работу устройства. Для обычных приложений используйте только `loadable_app::Api`.

Сборка

Один APP:

```
./tools/build_f401_bundle.sh \  
--profile mini-v3-a00 \  
--app-manifest path/to/MYAPP/app.mk61
```

Несколько приложений:

```
./tools/build_f401_bundle.sh \  
--profile mini-v3-a00 \  
--app-manifest apps/CLOCK/app.mk61 \  
--app-manifest apps/GAME/app.mk61
```

Число `--app-manifest` не ограничено искусственной константой. Можно также передать список через `MK61_APP_MANIFESTS`: разделитель `:` на macOS/Linux и `;` на Windows. Для одного Windows-пути вида `C:\...\app.mk61` разделитель не нужен.

```
MK61_APP_MANIFESTS=apps/CLOCK/app.mk61:apps/GAME/app.mk61 \  
./tools/mk61-firmware.cmd --mcu f401 --profile mini-v3-a00 --build
```

Переменная наследуется командами `--build` и `--upload`. Host-компилятор C++17 теперь используется и штатным ZX0-паковщиком; Bash и legacy-сборщик нужны только для manifest-сборки пользовательских APP. Без `MK61_APP_MANIFESTS` оба порта `mk61-firmware` используют канонический `build-gcc.cmd` и общий `system_apps`-сборщик. Комплект только с пользовательскими APP штатно собирается и тогда, когда FOCAL, TinyBASIC, WBMP viewer и CHIP-8 выключены.

Сборщик:

- 1 собирает resident с `-Os`;
- 2 извлекает реальный адрес overlay из resident ELF;
- 3 для каждого APP создаёт изолированный Arduino sketch только с SDK и его manifest-файлами;
- 4 компилирует APP с `-Os -flto`;
- 5 связывает только перечисленные source;
- 6 проверяет секции, точку входа и 20-КиБ лимит;
- 7 оптимально сжимает образ ZX0 и создаёт привязанный контейнер.

Результат:

```
binary/mk61s-M-mini-v3-lcd1602-a00-f401/  
├─ mk61s-M-mini-v3-lcd1602-a00-f401.bin  
├─ build.flags  
├─ build.apps  
├─ System/  
│   ├── FOCAL.APP  
│   ├── BASIC.APP  
│   ├── MARKDOWN.APP  
│   └── CHIP8.APP  
└─ Apps/  
    ├── CLOCK.APP  
    └── GAME.APP
```

System содержит только включённые compile-time ключами компоненты. На fullscreen bitmap backend при включённом Markdown WBMP.APP отсутствует и его функции входят в MARKDOWN.APP, включая профиль WS0010 100x16. Apps является полностью генерируемым каталогом: следующая сборка удаляет устаревшие пользовательские артефакты, которых больше нет в manifest-списке. build.apps хранит точный список созданных путей.

Прошивка и установка

Прошить согласованный resident:

```
mk61_APP_MANIFESTS=path/to/MYAPP/app.mk61 \  
./tools/mk61-firmware.cmd \  
--mcu f401 --profile mini-v3-a00 --upload
```

Если включены System APP, после перезапуска откройте на калькуляторе Меню → USB-диск и выполните:

```
./tools/mk61-firmware.cmd \  
--mcu f401 --profile mini-v3-a00 --install-apps
```

Этот второй шаг синхронизирует только канонические файлы из System: копирует включённые и удаляет FOCAL.APP, BASIC.APP, WBMP.APP, MARKDOWN.APP или CHIP8.APP, если соответствующий compile-time ключ выключен. Остальные файлы C5 он не трогает. Пользовательские файлы из Apps копируются вручную:

- 1 откройте на MK61s Меню → USB-диск;
- 2 создайте на диске MK61S C5 каталог /Apps, если он нужен;
- 3 скопируйте APP из того же комплекта;
- 4 дождитесь завершения копирования и выполните безопасное извлечение диска;
- 5 только после sync/eject нажмите ESC на MK61s.

То же можно сделать через правую панель tools/mks.cmd. APP видны в MKC, Проводнике и USB-диске как обычные файлы; атрибуты hidden, system и read-only не устанавливаются. MKC передаёт APP потоково через fsput и сверяет размер и контрольную сумму.

Что происходит при копировании многих файлов

C5 не резервирует фиксированный массив слотов APP. Количество ограничено общей ёмкостью flash, node-ID и каталогом; host-тест создаёт 200 APP, а отдельный VFAT-тест одним sync импортирует 25 APP.

Одна незавершённая USB-транзакция может содержать до 384 уникальных 512-байтных блоков staging, то есть 192 КиБ изменённых данных и метаданных. Это предел одной порции до SYNCHRONIZE CACHE, а не предел числа хранимых APP. Если хост пытается превысить его без sync, следующий блок отклоняется без частичной публикации дерева C5; тест заполняет все 384 позиции, проверяет отказ 385-й и восстановление старых файлов после reboot. Большие наборы максимальных APP копируйте порциями с безопасным извлечением/sync между ними; MKC и fsput фиксируют файлы последовательно.

USB-копирование большого набора не является одной общей транзакцией: при отключении питания часть уже завершённых файлов может быть новой, а файлы, до которых операция не дошла, — старыми или отсутствовать. Это нормальная семантика пакетного копирования и не означает повреждение файловой системы.

Для каждого публикуемого файла и изменения каталога действуют COW/WAL. Persistent staging переживает reset; после каждого моделируемого места обрыва повтор sync доводит пакет до согласованного результата. Новый или изменённый невалидный APP отклоняется на read-only preflight до удаления, перемещения или замены любого узла C5. Уже лежащий неизменённый APP не блокирует остальные операции после обновления resident; его совместимость всё равно проверяется при запуске. Если FAT повторно выделяет новому файлу освободившийся extent уменьшаемого APP, C5 атомарно отделяет этот extent после preflight; результат не зависит от порядка записей каталога. После реального обрыва снова подключите диск и повторите копирование всего набора.

Запуск

Короткое OK на APP в Проводнике загружает и запускает его. Терминальные команды эквивалентны:

```
open /Apps/MYAPP.APP  
run /Apps/MYAPP.APP
```

Переименование и перемещение не меняют контейнер. Не помещайте пользовательский APP под каноническим системным именем в /System.

Для не-APP файла Проводник берёт двухбайтовую сигнатуру из его типа и вызывает зарегистрированный обработчик. Поэтому `open /Games/PONG.ch8` загружает `/System/CHIP8.APP`, а не пытается исполнять ROM как APP. Аналогично `open /Docs/readme.md` использует обработчик T2 из `/System/MARKDOWN.APP`.

Диагностика

Ошибка	Причина или действие
<code>app/firmware mismatch / PUT_FIRMWARE</code>	Resident и APP взяты из разных комплектов.
<code>corrupt app / PUT_APP</code>	Неверный заголовок, размер, ZX0-поток или CRC; повторите копирование правильного файла.
<code>app not installed</code>	Путь не ведёт к совместимому APP нужного kind.
<code>another app is active</code>	Выполняется вложенная попытка загрузить APP.
ГРАФИКА / НЕДОСТУПНА	Обработчику нужен графический экран, но текущий backend не графический.
Неоднозначный обработчик	Уберите один из пользовательских APP с одинаковым <code>handled_type_magic</code> .
<code>undefined reference</code>	APP вызвал функцию вне API либо не перечислен нужный source.
<code>MK61 module does not fit the SRAM overlay</code>	Image + BSS превышают 20 КиБ.
<code>unsupported global constructors/destructors</code>	Уберите нетривиальные глобальные объекты.
Packager сообщает slot limit	Сжатый контейнер превышает 20 544 байта.

Перед публикацией сохраните вместе исходники, manifest, весь каталог комплекта и версию STM32 toolchain. Самостоятельный .APP без согласованного resident .bin не является воспроизводимым релизом.